

Operating Systems – 67808

Exercise 3: Virtual Memory – Hierarchical Page Tables

Before you start, don't forget to read the course guidelines!

1 Virtual Memory Review

Virtual memory is a memory-management model that allows processes to use more memory than is actually available on the host. This is achieved by mapping virtual (a.k.a. logical) addresses (what the process sees) into physical addresses (actual locations in the RAM).

Since the physical address space is significantly smaller than the virtual address space, not all memory space of all processes (or even of one process!) can fit in the RAM at the same time. Therefore, parts of the virtual memory must be stored in the hard drive until the next time they are used by their process.

Paging is an approach where the virtual address space is partitioned into fixed-sized contiguous blocks, called pages. The physical address space is similarly partitioned into blocks the same size as pages, called frames. At any given time, each page is either mapped to a frame in the physical memory or is stored in the hard drive. When a process tries to access a virtual memory address in a page that is not in the physical memory, that page must be brought into the physical memory (swapped in). If there are no unused frames, another page must be evicted from the physical memory (swapped out). The mapping between pages and frames is done using page tables. The naive implementation will have a big table where the number in the p^{th} row is the index of the frame to which the p^{th} page is mapped.

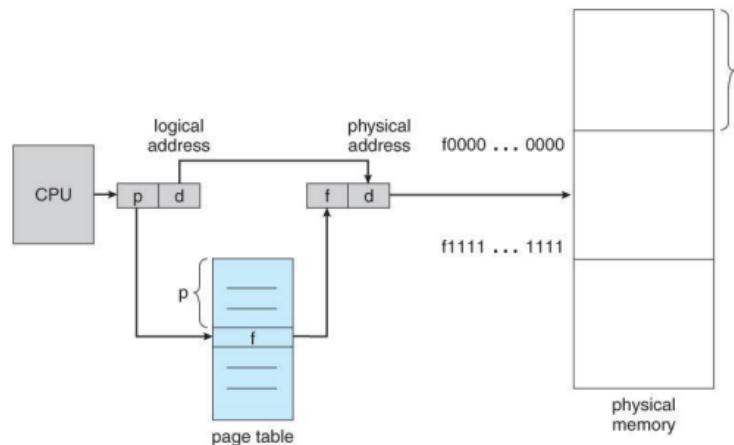


Figure 1: Virtual address to physical address mapping diagram

In this implementation each address is split into two parts: the offset and the page number. The offset (d in Figure 1) is the position within the page and hence doesn't need any translation. Its width in bits is $\log_2(\text{page_size})$.

The page number is the rest of the address, and its width is the remaining bits, or more

precisely:

$$\log_2(\text{virtual_memory_size}) - \log_2(\text{page_size})$$

The problem with a single page table is that it can be very wasteful in terms of physical memory consumption, usually there are many unused pages in the middle of the virtual address space which needlessly bloat the table, even to the point that it may not fit in the physical memory at all.

Hierarchical page tables use a tree of smaller tables to save memory. This effectively separates the translation into multiple steps. These tables reside in frames of the RAM just like any other page. Each row of the table points to a frame that contains either the next layer (the next table), or the relevant page itself (if it's the last, i.e., the lowest, layer). Each table can only contain a small number of rows, so there are only so many bits it can translate on its own, $\log_2(\text{number_of_rows})$ bits to be precise. In total the page tables are supposed to translate $\log_2(\text{virtual_memory_size}) - \log_2(\text{page_size})$ bits, so the depth of the page tables tree must be:

$$\left\lceil \frac{\log_2(\text{virtual_memory_size}) - \log_2(\text{page_size})}{\log_2(\text{number_of_rows})} \right\rceil$$

where $\lceil x \rceil$ is the ceil function.

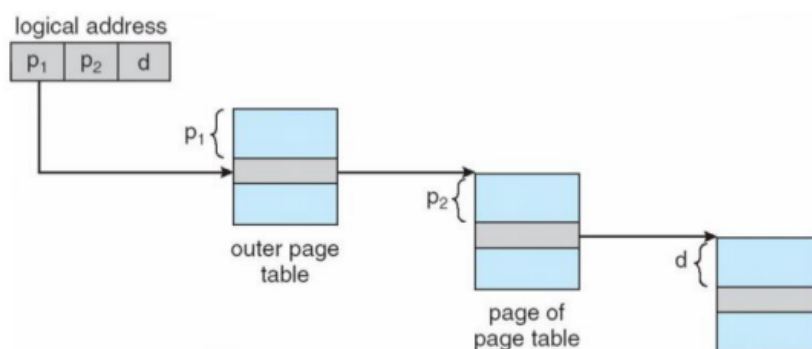


Figure 2: An example of address translation using a tree with 2 layers of tables

In this exercise you will implement a virtual memory interface using hierarchical page tables of arbitrary depth using simulated physical memory.

2 Task Overview

Your task is to implement the virtual memory API, as defined by the functions in `VirtualMemory.h`. You will do so by using the physical memory API in `PhysicalMemory.h`. An implementation of this API exists in `PhysicalMemory.cpp`. The file `MemoryConstants.h` contains many important constants such as the sizes of the memory spaces and the depth of the page tables tree. Make sure you understand the calculations within. Notice we will test your code with different memory sizes and trees of different depths, so your implementation must not rely on the specific values of constants in this file. Similar to how virtual address translation is done using the page tables in the RAM, you will also have to use the simulated physical memory to store your tables. Therefore, you are forbidden from using global variables or dynamic allocation. That includes dynamic allocation done by STL data structures. You cannot use `std::vector`, `std::map`, etc. This may sound difficult, but in fact it is quite possible. The next section details how. Note that

our implementation simulates the page tables of a single process. In reality there are multiple page table trees, one for each process.

3 Design

In our design, the root page table is always in frame 0 of the physical memory. Each row of the table (and of any other table) fits into a single word. Every table translates $\log_2(\text{page_size})$ bits, except the root table which translates the remaining bits. Translating an address therefore is quite simple: Assuming $2^4 = 16$ words in each page, a tree of depth 2 and that the address is in the physical memory, writing to the address `[101][0001][0110]` will take 3 steps:

```
PMread(0 + 5, &addr1) // first translation
PMread(addr1 * PAGE_SIZE + 1, &addr2) // second translation
PMwrite(addr2 * PAGE_SIZE + 6, value)
```

A read will be quite similar with only the last step different. The complexity arises when we consider that the virtual address may not be in the physical memory. But first how do we know if a page or any of its parent tables are in the physical memory? Since frame 0 always contains the root table (which is never evicted), no row in any table will ever point to frame 0. Therefore, we can just put 0 in rows whose page is currently swapped out. Now, if during any step of the translation we reach a row that contains the number 0, we know that this is not a real frame index and we have reached a page fault.

Now we have to swap the page back in and add it to the tree, creating tables above it, and performing eviction to other pages if necessary.

The full write into address `[101][0001][0110]` is therefore:

```
PMread(0 + 5, &addr1) // first translation
If (addr1 == 0) then
    Find an unused frame or evict a page from some frame. Suppose this frame number is f1
    Write 0 in all of its contents (only necessary if next layer is a table)
    PMwrite(0 + 5, f1)
    addr1 = f1
PMread(addr1 * PAGE_SIZE + 1, &addr2) // second translation
If (addr2 == 0) then
    Find an unused frame or evict a page from some frame. Suppose this frame number is f2.
    Make sure you are not evicting f1 by accident
    Restore the page we are looking for to frame f2 (only necessary pointing to a page)
    PMwrite(addr1 * PAGE_SIZE + 1, f2)
    addr2 = f2
PMwrite(addr2 * PAGE_SIZE + 6, value)
```

The last issues left are finding the exact frame to put our page in and evicting another page if necessary. We choose the frame by traversing the entire tree of tables in the physical memory while looking for one of the following, prioritized from 1 (highest) to 3 (lowest):

1. A frame containing an empty table – where all rows are 0. We don't have to evict it, but we do have to remove the reference to this table from its parent.
2. An unused frame – When traversing the tree, keep a variable with the maximal frame Index referenced from any table we visit. Since we fill frames in order, all used frames are contiguous in the memory, and if `max frame index + 1 < NUM_FRAMES` then we know that the frame in the index (`max frame index + 1`) is unused.

3. If all frames are already used, then a page must be swapped out from some frame in order to replace it with the relevant page (a table or actual page). The frame that will be chosen is the frame containing a page p such that the cyclical distance: $\min\{\text{NUM_PAGES} - |\text{page swapped in} - p|, |\text{page swapped in} - p|\}$ is maximal. If there are several pages with the same distance, the one with the lowest page number among them will be chosen. This page must be swapped out before we use the frame, and we also have to remove the reference to this page from its parent table.

Note: According to this design, `VMInitialize()` only has to clear frame 0.

4 Example for eviction

Say we have 8 pages, pages 1, 5 and 7 are occupied, and page 2 is inserted, we get distances 1, 3 and 3, respectively, and select page 5 since it has the lowest page number among the pages with maximal cyclical distance.

1. Visualizing the Cyclical System

Imagine the 8 pages arranged in a circle, you can travel between pages in two directions: **clockwise** (forward) or **counter-clockwise** (backward). The "cyclical distance" is simply the shortest path between two points along this circle. In this example, we have $\text{NUM_PAGES} = 8$ and $\text{page swapped in} = 2$.

2. Calculating the Distances

We need to find the cyclical distance from the inserted page (2) to each of the occupied pages (1, 5, and 7). We do this by calculating the distance in both directions and taking the smaller number.

- **Distance to Page 1:**
 - Direct distance: $|2 - 1| = 1$ step.
 - Wrap-around distance: $8 - 1 = 7$ steps.
 - Shortest cyclical distance: $\min(1, 7) = 1$.
 - (*Path: $2 \rightarrow 1$*)
 - **Distance to Page 5:**
 - Direct distance: $|2 - 5| = 3$ steps.
 - Wrap-around distance: $8 - 3 = 5$ steps.
 - Shortest cyclical distance: $\min(3, 5) = 3$.
 - (*Path: $2 \rightarrow 3 \rightarrow 4 \rightarrow 5$*)
 - **Distance to Page 7:**
 - Direct distance: $|2 - 7| = 5$ steps.
 - Wrap-around distance: $8 - 5 = 3$ steps.
 - Shortest cyclical distance: $\min(5, 3) = 3$.
 - (*Path backwards: $2 \rightarrow 1 \rightarrow 8 \rightarrow 7$*)
- The distances are **1, 3, and 3**, respectively.

3. Finding the Maximal Distance

The algorithm dictates that we must first find the pages with the **maximal cyclical distance** from the newly inserted page.

- The set of all distances is $\{1, 3, 3\}$.

- The maximum value in this set is 3.
- The occupied pages that are a distance of 3 away are **Page 5** and **Page 7**.

4. Applying the Tie-Breaking Rule

We now have a tie between Page 5 and Page 7. To resolve this, we simply compare the page numbers of our candidates:

$$5 < 7$$

Because 5 is strictly less than 7, **Page 5** is selected.

5 Your Assignment

- (not for submission) answer on the following questions:
 - With virtual address width of 10 bit, physical width of 6 bits, offset of 3 bit: What is the page table depth? What is the maximal virtual pages one can write without triggering an eviction? What is the minimal number of pages?
 - When looking to evict a page, is that possible that we will find 2 distinct table entries with all entries zeroed-out? Why or why not?
 - Why do we choose the maximal cyclical distance as eviction criteria? What kind of memory access pattern will benefit from this decision? Which other decision we could have made?
- Implement the functions defined in `VirtualMemory.h` using the design described above.
- Your code must support different memory space and page sizes in `MemoryConstants.h`. You can assume these are all valid.
- You must not use global variables or dynamic allocations.
- Do not submit any of the `PhysicalMemory` files.
- Do not include `<iostream>` in the submitted files as it defines global variables.

6 School solution

We provide a school solution for reference.

- To run the school solution with your own cpp files, run:
`~os/ex/ex3/run_school_solution main.cpp`
- To run the school solution and send CLI arguments to the resulting program, run
`~os/ex/ex3/run_school_solution main.cpp -- ARG1 ARGS2 ...`
- To run the school solution with custom memory constants, create a file called *overrides.h*:

```
#define OFFSET_WIDTH 3
#define PHYSICAL_ADDRESS_WIDTH 5
#define VIRTUAL_ADDRESS_WIDTH 6
```

Then run

```
~os/ex/ex3/run_school_solution overrides.h main.cpp
```

And see the constants applied to your program. This file is automatically included, so you do not have to include it in your code.

- For example, cd into the supplied directory from Moodle, then try to run:

```
~os/ex/ex3/run_school_solution tests/test0_sanity.cpp
```

And

```
~os/ex/ex3/run_school_solution tests/test0_sanity.cpp example/overrides.h
```

Notice how the address in the second line changes between the runs, since the size of the virtual memory was changed by the overrides.

7 Tips

1. There is a lot of bit twiddling in this exercise. These calculations are very easy to get wrong, so try to separate them into small functions where possible and verify that they do what you think they do.
2. Pay special attention to variable types and signed-ness.
3. When swapping a page in, you may also need to create new tables for its ancestors in the tree. Make sure that you create these from top to bottom and that you don't accidentally evict a table you have just created.
4. Run the school solution on several main files and understand its output.
5. Test your code with different memory constants and different tree depths. To do so, use `overrides.h` or compile it with user-defined macros, for example:

```
g++ -DOFFSET_WIDTH=3 -DPHYSICAL_ADDRESS_WIDTH=5 -DVIRTUAL_ADDRESS_WIDTH=6 *.cpp
```

8 Submission

Submit a tar file containing the following:

1. A README file. The README should be structured according to the course guidelines. In order to be compliant with the guidelines.
2. The source files for your implementation of the library.
3. The submission MUST pass the presubmit before being submitted. Submissions that does not pass are considered invalid, and might result 0 grade for the entire exercise.